What is kube-apiserver?

Professional Answer

The API Server is the central management component that exposes Kubernetes APIs, validates requests, enforces authentication and authorization, and acts as the gateway to the cluster.

Unprofessional Answer

API server talks to Kubernetes.

Why is API Server called the “front door” of Kubernetes?

Professional Answer

All internal and external communication in Kubernetes goes through the API Server, making it the single entry point for cluster interactions.

Unprofessional Answer

Because everything goes through it.

What is kube-apiserver and what does it do?

kube-apiserver is the central management component of Kubernetes that exposes the REST APIs.
All external and internal communication in the cluster goes through the API server.
It handles authentication, authorization, admission control, and request validation.
The API server stores and retrieves cluster state from etcd.
It is stateless and can be horizontally scaled for high availability.

Kubernetes Internals – Component-by-Component

1. kube-apiserver (The Brain Gateway)

What it is internally

- A **stateless REST API service**
- Written in **Go**
- Fronted by HTTPS
- Can be horizontally scaled

Internal Responsibilities

1. **Authentication**
    - Validates *who* is making the request
    - Supports:
        - Certificates
        - Tokens
        - OIDC
        - Service Accounts
2. **Authorization**
    - Decides *what* the user can do
    - Uses:
        - RBAC
        - ABAC
        - Webhooks
3. **Admission Controllers**
    - Intercepts requests **after auth**
    - Can:
        - Validate objects
        - Mutate objects
    - Examples:
        - LimitRanger
        - ResourceQuota
        - PodSecurity

        - MutatingWebhook

4. **API Validation**
    - Validates schema
    - Ensures object correctness
5. **Persistence**
    - Writes state to **etcd**
    - Reads state from etcd

What is kube-apiserver?

The kube-apiserver is the **central management component** of Kubernetes.
It exposes the **Kubernetes API** over REST/HTTPS.
All cluster operations go through it.
It validates, authenticates, authorizes, and persists data.
No other component talks directly to etcd.

Why kube-apiserver is critical

- Single entry point to the cluster
- Controls state transitions
- Enforces security policies
- Mediates between users and control plane
- Failure = control plane outage

If kube-apiserver is down, Kubernetes is down.

Who talks to kube-apiserver?

Component	Interaction
kubectl	REST calls
Scheduler	Watches Pods
Controller Manager	Watches desired vs actual
kubelet	Pod status updates
etcd	Read/write state

Rule: Nothing bypasses the API Server.

What is Kubernetes Scheduler?

Professional Answer

The Scheduler assigns pods to nodes based on resource availability, constraints, affinity rules, and policies.

Unprofessional Answer

Scheduler decides where pods run.

What is the role of kube-scheduler?

The kube-scheduler is responsible for assigning pods to appropriate worker nodes.
It watches for newly created pods that do not have a node assigned.
The scheduler evaluates nodes based on resource availability and scheduling constraints.
It considers factors like CPU, memory, taints, tolerations, and affinity rules.
After selection, it updates the pod specification with the chosen node name.

What factors does the scheduler consider?

Professional Answer

- CPU & memory requests
- Node taints & tolerations
- Pod affinity/anti-affinity

- Node selectors
- Topology constraints

Unprofessional Answer

Resources and rules.

kube-scheduler (Scheduling Internals)

What it does internally

Scheduler works in **two main phases**:

Phase 1: Filtering (Predicates)

Eliminates nodes that cannot run the pod:

- Insufficient CPU/memory
- Taints not tolerated
- NodeSelector mismatch
- Volume conflicts

Phase 2: Scoring (Priorities)

Scores remaining nodes:

- Least loaded node
- Pod affinity
- Topology spread
- Custom scoring plugins

Final Step

- Highest score node selected
- Writes node name to PodSpec via API server

Important

Scheduler **never runs containers**, only assigns nodes

What is kube-scheduler?

Professional Answer

kube-scheduler is a control plane component responsible for selecting a suitable worker node for a Pod.
It watches for Pods that do not have a node assigned.
It evaluates nodes based on resources and constraints.
It assigns the Pod to the best-fit node.
It does not run Pods itself.

Weak Answer

Scheduler decides where pods go.

When does kube-scheduler act?

Professional Answer

kube-scheduler acts only when a Pod is created without a node assignment.
It does not manage running Pods.
It does not reschedule Pods automatically.
Once a Pod is bound to a node, the scheduler is done.
Re-scheduling happens only after Pod recreation.

Does kube-scheduler run on worker nodes?

Professional Answer

No, kube-scheduler runs on control plane nodes.
It runs as a static Pod or system service.
Multiple scheduler instances can run for HA.
Only one instance becomes leader at a time.
Leader election prevents double scheduling.

What is Controller Manager?

Professional Answer

The Controller Manager runs control loops that continuously monitor cluster state and ensure the actual state matches the desired state.

Unprofessional Answer

Controller manages Kubernetes.

Examples of Kubernetes controllers

Professional Answer

- Node Controller
- ReplicaSet Controller
- Deployment Controller
- Endpoint Controller

Unprofessional Answer

Deployment controller.

What does kube-controller-manager do internally?

kube-controller-manager runs multiple controllers that manage different cluster resources.
Each controller continuously compares the desired state with the actual state.
If a mismatch is detected, corrective actions are taken automatically.
Examples include ReplicaSet, Node, Deployment, and Endpoint controllers.
This reconciliation mechanism enables Kubernetes self-healing behavior.

kube-controller-manager (Reconciliation Engine)

Internal Design

- Runs **multiple controllers**
- Each controller is a **control loop**

Control Loop Logic

1. Watch desired state (API server)
2. Compare with actual state
3. Take corrective action
4. Repeat forever

Key Controllers Internals

- **ReplicaSet Controller**
    - Creates/deletes pods
- **Node Controller**
    - Detects node failure
- **Deployment Controller**
    - Manages rolling updates
- **Endpoint Controller**
    - Updates service endpoints

Core Principle

Kubernetes is a **reconciliation-based system**

What is kube-controller-manager?

kube-controller-manager is a **control-plane component** that runs multiple **controllers**.
Controllers continuously watch cluster state and **reconcile actual state to desired state**.
It operates on the **control loop pattern**.
It does not run workloads.
It ensures Kubernetes is self-healing.

Why kube-controller-manager exists

Kubernetes is **declarative**, not imperative.
Users define **desired state**.
Controllers enforce that state continuously.
Without controllers, Kubernetes would not auto-heal.
This component is the **brain of reconciliation**.

How kube-controller-manager works

Each controller follows this loop:

1. Watch API Server
2. Compare desired vs current state

3. Take corrective action
4. Update API Server

This loop never stops.

What happens if kube-controller-manager goes down?

Feature	Impact
Existing Pods	Continue running
Auto-healing	Stops
Scaling	Stops
Node eviction	Stops
Jobs	Fail silently

Cluster becomes **non self-healing**.

---

4. Important controllers inside kube-controller-manager

1. Node Controller

- Monitors node health
- Marks nodes NotReady
- Evicts Pods when node is unhealthy

---

2. Replication Controller

- Ensures correct Pod count
- Recreates deleted Pods
- Maintains availability

---

3. Deployment Controller

- Manages ReplicaSets
- Handles rolling updates
- Ensures deployment strategy

---

4. StatefulSet Controller

- Maintains Pod identity
- Handles ordered deployment
- Manages stable storage

---

5. DaemonSet Controller

- Ensures one Pod per node
- Handles node join/leave
- Used for agents (logging, monitoring)

---

6. Job & CronJob Controller

- Manages batch workloads
- Handles retries and failures
- Schedules periodic jobs

---

7. Endpoint Controller

- Updates Service endpoints
- Tracks healthy Pods
- Keeps traffic routing accurate

---

8. Namespace Controller

- Manages namespace lifecycle

- Handles deletion & cleanup
- Removes dependent resources

---

9. ServiceAccount & Token Controller

- Creates default service accounts
- Manages token secrets
- Enables Pod authentication

What is Cloud Controller Manager?

Professional Answer

Cloud Controller Manager integrates Kubernetes with cloud-provider APIs for managing load balancers, volumes, and node lifecycle.

Unprofessional Answer

It manages cloud things.

Why is cloud-controller-manager used in Kubernetes?

cloud-controller-manager integrates Kubernetes with cloud provider services.
It manages cloud-specific resources like load balancers, volumes, and node lifecycle.
This separation keeps Kubernetes core cloud-agnostic.
It communicates directly with cloud provider APIs such as AWS, GCP, or Azure.
It allows Kubernetes to run consistently across different cloud environments.

cloud-controller-manager (Cloud Internals)

Why it exists

- Separates cloud logic from core Kubernetes

Internal Controllers

1. **Node Controller**
    - Cloud node lifecycle
2. **Service Controller**

    - Creates cloud load balancers
3. **Route Controller**
    - Manages network routes
4. **Volume Controller**
    - Attaches/detaches disks

Works with

- AWS APIs
- GCP APIs
- Azure APIs

What is Cloud Controller Manager?

Cloud Controller Manager is a **Kubernetes control-plane component** that integrates Kubernetes with **cloud provider APIs**.
It allows Kubernetes to manage cloud resources **without hard-coding cloud logic into core components**.
CCM runs cloud-specific controllers.
It communicates with both **kube-apiserver** and **cloud APIs**.
Without CCM, Kubernetes cannot manage cloud-native resources properly.

Why Cloud Controller Manager was introduced?

Earlier, cloud logic was inside:

- kube-controller-manager
- kubelet

This caused:

- Tight cloud coupling
- Hard upgrades
- Security risks

CCM **decouples cloud logic** from Kubernetes core.

This is a **design evolution question** interviewers love.

Where does CCM run?

- As a **control plane component**
- Usually as a **pod** (static pod or deployment)
- Runs only in **cloud environments**
- Not needed for bare-metal clusters

Managed services (EKS, GKE, AKS) run it for you.

What is etcd?

Professional Answer

etcd is a distributed, highly available key-value store used to persist the entire cluster state, including configuration, metadata, and secrets.

Unprofessional Answer

etcd stores Kubernetes data.

What is etcd in Kubernetes architecture?

etcd is a distributed and consistent key-value store used to store the entire cluster state.
It acts as the single source of truth for configuration, metadata, and secrets.
etcd uses the Raft consensus algorithm to ensure strong consistency.
Only the kube-apiserver communicates directly with etcd.
If etcd data is lost, the Kubernetes cluster cannot be recovered.

Why is etcd critical to Kubernetes?

Professional Answer

etcd is the source of truth for the cluster; losing etcd means losing cluster state, configurations, and workloads.

Unprofessional Answer

Without etcd, Kubernetes won't work.

2. etcd (Cluster Memory Internals)

What it is internally

- Distributed key-value store
- Uses **RAFT consensus**
- Strong consistency (CP system)

Internal Working

- Stores everything as key-value pairs
- Example:
- /registry/pods/default/nginx

Key Internals

1. **Leader Election**
    - One leader
    - Followers replicate data
2. **Write Path**
    - Request → Leader → Majority acknowledgment → Commit
3. **Watch Mechanism**
    - API server and controllers watch etcd for changes
    - Enables event-driven architecture

Why etcd is critical

- Lose etcd → lose cluster state
- Backups are mandatory in production

What is etcd in Kubernetes?

Answer:
etcd is a **distributed, consistent, and highly available key-value store** used by Kubernetes to store **all cluster state**.
It acts as the **single source of truth** for Kubernetes.
Every object—Pods, Nodes, Secrets, ConfigMaps, Deployments—is stored in etcd.
If etcd is lost, the cluster loses its state.
Kubernetes control plane components continuously read from and write to etcd.

Why Kubernetes uses etcd?

Answer:
Kubernetes needs **strong consistency**, not eventual consistency.
etcd provides **linearizable reads and writes** using the **Raft consensus algorithm**.
It ensures all control plane components see the same cluster state.
etcd supports **watch functionality**, enabling real-time updates.
This makes scheduling, reconciliation, and self-healing possible.

Where does etcd run?

Answer:
In production clusters, etcd runs as **static pods** on control plane nodes.
In managed Kubernetes (EKS/GKE/AKS), etcd is fully managed by the cloud provider.
It typically listens on ports **2379 (client)** and **2380 (peer)**.

etcd is isolated from worker nodes for security reasons.
Only control plane components can access it.

What data is stored in etcd?

Answer:
etcd stores **cluster metadata**, not application data.
Examples include Pods, ReplicaSets, Services, Secrets, RBAC policies, and Node info.
Secrets are stored **base64-encoded**, not encrypted by default.
Encryption at rest must be explicitly enabled.
etcd does not store container logs or runtime data.

How control plane components interact with etcd

Answer:

- **API Server** is the **only component** that directly talks to etcd
- **Scheduler** reads pod specs via API Server
- **Controller Manager** watches state changes via API Server
- **kubelet** never talks to etcd directly
  This design enforces security and consistency.

What happens if etcd goes down?

Answer:

- No new Pods can be scheduled
- kubectl commands fail
- Existing workloads continue running
- Controllers cannot reconcile state
- Cluster becomes **read-only or completely unusable**